



CSE3433 SOFTWARE ARCHITECTURE

PROGRAMME: COMPUTER SCIENCE (SOFTWARE ENGINEERING)

SEMESTER II 2025/2026

PROJECT TITLE:

SMARTLEARN: CLOUD-NATIVE LEARNING MANAGEMENT SYSTEM (LMS)

LECTURER: DR. MOHAMAD NOR HASAN

NO.	GROUP MEMBERS	MATRIC NO.
1.	PUTERI IREENE HAZWANI BINTI MUHAMAD HAIZAR	S75702
2.	NURAIN NAZIRAH BINTI ABDULLAH	S75415
3.	NURUL ALESSA BINTI MOHD NIZAM	S76278

Table of Content

Table of Content.....	1
1.0 System Overview.....	3
2.0 Problem Description.....	3
3.0 System Objective.....	4
4.0 Functional Requirements.....	5
5.0 Non Functional Requirements.....	6
6.0 Architecture Style Explanation.....	7
6.1 Overview of Architectural Style.....	7
6.2 Layered (3-Tier) Architecture.....	7
6.2.1 Presentation Layer.....	7
6.2.2 Application Layer.....	7
6.2.3 Data Layer.....	8
6.3 Cloud-Native Architecture.....	8
6.4 Modular Design and Separation of Concerns.....	9
6.5 Justification of Architectural Style.....	9
7.0 Service / Component Identification.....	10
7.1 User Management Module.....	10
Functions:.....	10
Associated Components:.....	10
7.2 Course Management Module.....	10
Functions:.....	10
Associated Components:.....	11
7.3 Assignment Management Module.....	11
Functions:.....	11
Associated Components:.....	11
7.4 Grading Module.....	11
Functions:.....	11
Associated Components:.....	12
7.5 Supporting Components.....	12
7.5.1 Security Component.....	12
7.5.2 Notification Component.....	12
7.6 Summary of Module Interaction.....	12
7.7 Design Justification.....	13
8.0 System Boundaries.....	14
8.1 User Management Module.....	14
8.2 Course Management Module.....	14
8.3 Assignment Management Module.....	15
8.4 Grading Module.....	15
9.0 Communication Design.....	16

9.1 Communication Approach.....	16
9.2 Interaction Mapping.....	16
9.2.1 User Authentication Flow.....	16
9.2.2 Course Enrollment Flow.....	16
9.2.3 Assignment Submission Flow.....	16
9.2.4 Grading Flow.....	16
9.3 System Communication Summary.....	17
10.0 Data Ownership.....	18
10.1 Data Ownership Table.....	18
10.2 Supporting Components.....	18
10.3 Simple Principle.....	18
11.0 Architecture Diagrams.....	19
11.1 Context Diagram.....	19
11.2 Component/Service Architecture Diagram.....	20
11.3 Interaction Diagram.....	22
11.3.1 User Management Module.....	22
11.3.2 Course Management Module.....	22
11.3.3 Assignment Management Module.....	23
11.3.4 Grading Module.....	24
11.4 Deployment Diagram.....	25
12.0 Team Member Roles.....	27

1.0 System Overview

The proposed system is a Cloud-Native Learning Management System (LMS) named **SMARTLEARN**, designed to support teaching and learning activities in an online environment. The system allows students to enroll in courses, access learning materials, and submit assignments, while lecturers can manage courses and evaluate student submissions.

The system is built using a **Cloud-Native Layered (3-Tier Architecture)** consisting of Presentation Layer, Application Layer, and Data Layer. This architecture ensures scalability, maintainability, and clear separation of system responsibilities.

The system is deployed on cloud infrastructure to support accessibility, performance, and concurrent user access.

2.0 Problem Description

In many educational institutions, existing Learning Management Systems (LMS) have several limitations, especially when a large number of users access the system at the same time. This often causes slow performance, delays in loading pages, and a poor user experience for both students and lecturers.

In addition, some LMS are not scalable and lack flexibility when adding new features or handling increasing numbers of users. This makes it difficult for the system to support growing academic needs.

Furthermore, many traditional LMS are built using monolithic architecture, where all components are combined into a single system. This makes the system harder to maintain, update, and improve. Any small change may affect the whole system and can take a long time to complete. As a result, system updates may interrupt ongoing academic activities and reduce overall efficiency. Therefore, there is a need to develop a modern LMS that uses cloud-native principles and layered architecture to improve system performance, scalability, flexibility, and user experience.

3.0 System Objective

The objectives of this project are:

3.1 Cloud-native LMS design with scalability and accessibility

- The system shall be designed as a cloud-native Learning Management System (LMS).
- The system must support many users accessing the platform at the same time without affecting performance.
- The system shall be accessible anytime and anywhere through an internet connection.

3.2 Implementation of layered (3-tier) architecture

- The system shall be developed using a layered (3-tier) architecture.
- The system must separate the presentation layer, application layer, and data layer.
- This will improve system structure, organization, and ease of maintenance.

3.3 Provision of basic LMS functionalities

- The system shall provide essential LMS features such as course management and assignment handling.
- The system must allow lecturers to manage courses and students to access course materials.
- The system shall support assignment submission and management in an organized way.

3.4 Modular design and separation of concerns

- The system shall be developed using modular design principles.
- Each module must handle specific functions independently.
- This will make the system easier to maintain, update, and improve in the future.

4.0 Functional Requirements

4.1 User Management Module

- The system shall allow users to register with an email and password.
- The system shall assign them a role either as a student or a lecturer.

4.2 Course Management Module

- The lecturer shall be able to create new courses by defining a title , description and course code (example : CSF3433)
- The lecturer must have the ability to edit course details or delete it.
- The student shall be able to join the course by entering the course code that is provided by their lecturer.
- The system must display all courses that have been joined by students in their personal dashboard.

4.3 Assignment Management Module

- The lecturer shall set up assignments submission links with specific instruction and submission deadlines.
- The students must be able to upload their assignment in digital files.

4.4 Grading Module

- The system must show real-time status labels for assignment such as “Not Graded” , “Draft” or “Submitted”
- The system shall update the students’ grade view when the lecturer key-in their grades.

5.0 Non Functional Requirements

5.1 Performance

The system must ensure all pages and course materials load within 2 seconds to provide a smooth experience for students and lecturers. This ensures that users can navigate between courses and assignments without frustrating delays or lag.

5.2 Availability

The platform shall maintain 99.9% uptime to ensure learning materials and assignment links are accessible 24/7 without interruption. Any necessary maintenance should be scheduled during off-peak hours to minimize the impact on academic activities.

5.3 Scalability

The cloud infrastructure must automatically scale its resources to handle at least 1,000 concurrent users during high-traffic periods like final submission deadlines. This prevents the system from slowing down or crashing when a large number of students log in at the same time.

5.4 Security

All user data and grades must be protected through strong encryption and strict role-based access to prevent unauthorized viewing or tampering. The system must also ensure that students can only access the specific courses they are officially enrolled in.

5.5 Usability

The interface must be fully responsive so that users can easily access all features on desktops, tablets, and smartphones. The design should be intuitive enough that a new user can find their course and submit an assignment with minimal guidance.

6.0 Architecture Style Explanation

6.1 Overview of Architectural Style

The SMARTLEARN system uses a **Cloud-Native Layered Architecture (3-Tier Architecture)** together with a **modular design**. This means the system is divided into different layers and parts, where each part has its own job.

The system has three main layers:

- Presentation Layer
- Application Layer
- Data Layer

By separating the system like this, it becomes easier to manage, fix, and improve. It also allows many users to use the system at the same time without slowing down.

6.2 Layered (3-Tier) Architecture

6.2.1 Presentation Layer

This is the part that users can see and interact with. It is accessed through a web browser on a phone, tablet, or computer.

This layer is responsible for:

- Showing pages (login, dashboard, courses)
- Taking user input (login details, assignment upload)
- Sending requests to the system
- Displaying results (grades, course list)

6.2.2 Application Layer

This layer is the “brain” of the system. It processes all requests from users and controls how the system works.

It includes:

- User Management Module
- Course Management Module
- Assignment Management Module
- Grading Module

This layer handles tasks such as:

- Checking login details
- Managing courses
- Handling assignment submissions
- Updating grades

6.2.3 Data Layer

This layer stores all the data of the system.

It is responsible for:

- Saving user information (student and lecturer)
- Saving course details
- Storing assignments and submissions
- Keeping grades and status

The system uses a database (e.g., MySQL) to store this information safely.

6.3 Cloud-Native Architecture

SMARTLEARN is built using cloud technology. This means the system is hosted online instead of on a single computer.

Benefits include:

- **Scalability**
The system can handle many users at the same time (at least 1000 users). If more users join, the system can increase its resources automatically.
- **Availability**
The system is available almost all the time (99.9% uptime), so users can access it anytime.
- **Performance**
Pages load quickly (within 2 seconds), so users do not experience delays.
- **Load Balancing**
The system shares the workload across servers to avoid crashing.

The system is deployed using **Render**, a cloud platform that supports web application hosting and automatic deployment. This allows the system to be accessible online and supports continuous deployment and scalability.

In addition, the system is developed and designed using several tools and technologies. The user interface is built using **HTML, CSS, and JSP**, while the backend logic is developed using **Java**. The system uses **MySQL** as the database to store all data.

For system design, tools such as **Draw.io (Diagrams.net)** and **PlantUML** are used to create architecture and UML diagrams. Email notifications are handled through a notification component integrated with an email service.

These tools and technologies support the cloud-native approach and help ensure the system is scalable, reliable, and easy to maintain.

6.4 Modular Design and Separation of Concerns

The system is divided into smaller parts called modules. Each module has its own function.

Modules in SMARTLEARN:

- User Management
- Course Management
- Assignment Management
- Grading

Benefits:

- Easier to fix problems
- Easier to add new features
- Changes in one module will not affect others

6.5 Justification of Architectural Style

This design is suitable because:

- It solves problems found in old LMS systems
- It can handle many users at the same time
- It is easier to maintain and update
- It provides better performance and user experience

7.0 Service / Component Identification

The SMARTLEARN system is decomposed into four main functional modules based on a **Cloud-Native Layered (3-Tier Architecture)**. Each module represents a core system capability and is implemented within the Application Layer, supported by the Presentation and Data layers. In addition to core modules, the system includes supporting components such as Security and Notification services to handle system-level operations.

7.1 User Management Module

This module is responsible for handling all user-related operations in the system.

Functions:

- User registration (Student and Lecturer)
- User login and authentication
- Role-based access control
- Profile management

Associated Components:

- Login Interface (Presentation Layer)
- Authentication Service (Application Layer)
- User Database (Data Layer)

7.2 Course Management Module

This module manages all course-related activities within SMARTLEARN.

Functions:

- Create, update, and delete courses (Lecturer)

- Manage course information (title, description, course code)
- Course enrollment (Student)
- Display enrolled courses

Associated Components:

- Course Interface (Presentation Layer)
- Course Management Service (Application Layer)
- Course Database (Data Layer)

7.3 Assignment Management Module

This module handles assignment creation, submission, and tracking.

Functions:

- Create assignments with instructions and deadlines (Lecturer)
- Submit assignments (Student)
- Upload and store assignment files
- Track submission status

Associated Components:

- Assignment Interface (Presentation Layer)
- Assignment Service (Application Layer)
- Assignment Database (Data Layer)

7.4 Grading Module

This module manages evaluation and grading of student submissions.

Functions:

- Lecturer assigns marks for submissions
- Update grading status (Draft / Submitted / Graded)

- View grades by students
- Maintain grading records

Associated Components:

- Grade Interface (Presentation Layer)
- Grade Management Service (Application Layer)
- Grade Database (Data Layer)

7.5 Supporting Components

This system includes supporting components that assist the core modules in handling system-level functions.

7.5.1 Security Component

Handles authentication and authorization. It ensures that only valid users can access the system and perform actions based on their roles.

7.5.2 Notification Component

Handles system notifications such as email alerts for assignment submission and grade updates. It is integrated with external email services (AWS SES).

7.6 Summary of Module Interaction

All modules interact through the Application Layer, which acts as the core processing layer. The flow is:

- User interacts via Presentation Layer
- Request is processed by respective module in Application Layer
- Data is stored/retrieved from Data Layer
- Response is returned to user interface

7.7 Design Justification

The system is divided into modules to ensure:

- Clear separation of responsibilities
- Easier maintenance and debugging
- Scalability in cloud environment
- Independent development of each module
- Alignment with cloud-native architecture principles

8.0 System Boundaries

8.1 User Management Module

This module is responsible for handling all user-related activities in the SMARTLEARN system. It ensures that users are properly authenticated and assigned correct roles.

DOES:

- User registration (student and lecturer)
- User login and authentication
- Role-based access control
- Profile management

DOES NOT:

- Manage course information
- Handle assignment submission
- Process grading activities

8.2 Course Management Module

This module manages all course-related operations such as creation and enrollment.

DOES:

- Create, update, and delete courses
- Manage course details (title, description, course code)
- Allow student enrollment using course code
- Display enrolled courses

DOES NOT:

- Handle assignment submission
- Perform grading functions
- Manage user authentication

8.3 Assignment Management Module

This module is responsible for handling assignment creation and submission processes.

DOES:

- Create assignments with instructions and deadlines
- Allow students to upload assignment files
- Track submission status
- Store submission data

DOES NOT:

- Calculate or assign grades
- Manage course enrollment
- Handle user authentication

8.4 Grading Module

This module manages evaluation and grading of student submissions.

DOES:

- Assign marks for submitted assignments
- Update grading status (Draft / Submitted / Graded)
- Display grades to students
- Store grading records

DOES NOT:

- Handle file submission
- Manage course content
- Handle user authentication

9.0 Communication Design

9.1 Communication Approach

The SMARTLEARN system uses **synchronous REST API communication** between Presentation Layer and Application Layer services. All core business logic is handled within individual micro-style services inside a layered architecture.

Security validation is handled through a **Security Component**, while system notifications are processed using a **Notification Component integrated with external email services (AWS SES)**.

9.2 Interaction Mapping

9.2.1 User Authentication Flow

- Web Application (SPA) / Mobile Application → User Service
- User Service → Security Component
- Security Component → User Database
- Authentication response → Web Application (SPA) / Mobile Application

9.2.2 Course Enrollment Flow

- Web Application (SPA) / Mobile Application → Course Service
- Course Service → Security Component (authorization check)
- Course Service → Course Database
- Enrollment confirmation → Web Application (SPA) / Mobile Application

9.2.3 Assignment Submission Flow

- Web Application (SPA) / Mobile Application → Assignment Service
- Assignment Service → Assignment Database (store submission)
- Assignment Service → Notification Component
- Notification Component → Email Service (AWS SES)
- Submission confirmation → Web Application (SPA) / Mobile Application

9.2.4 Grading Flow

- Web Application (SPA) / Mobile Application (Lecturer Interface) → Grading Service
- Grading Service → Grade Database

- Grading Service → Notification Component
- Notification Component → Email Service (AWS SES)
- Updated grade notification → Web Application (SPA) / Mobile Application (Student Interface)

9.3 System Communication Summary

All system interactions in SMARTLEARN originate from either the Web Application (SPA) or Mobile Application, which act as the Presentation Layer. These applications communicate with backend services via REST APIs in a synchronous request-response model.

Supporting components such as the Security Component handle authentication and authorization, while the Notification Component manages external communication through email services (AWS SES).

General flow:

Web Application (SPA) / Mobile Application → Application Services → Supporting Components → Data Layer → Response to Web Application (SPA) / Mobile Application

10.0 Data Ownership

Each service in SMARTLEARN manages its own data independently. This ensures loose coupling, better scalability, and clear separation of concerns.

10.1 Data Ownership Table

Module	Service	Database Owned	Data Stored
User Management	User Service	User Database	User details, login info, roles (student / lecturer)
Course Management	Course Service	Course Database	Course info, course code, and enrollment data
Assignment Management	Assignment Service	Assignment Database and File Storage	Assignment details and student submissions
Grading Module	Grading Service	Grade Database	Marks, grading status, and results

Table 1.0: Data Ownership Table

10.2 Supporting Components

Component	Data Role
Security Component	Does not store data, only checks login and access
Notification Component	Does not store data, only sends email alerts

Table 2.0: Supporting Components

10.3 Simple Principle

- Every service has its own database
- No service is allowed to directly access another service database
- All communication must go through APIs

11.0 Architecture Diagrams

11.1 Context Diagram

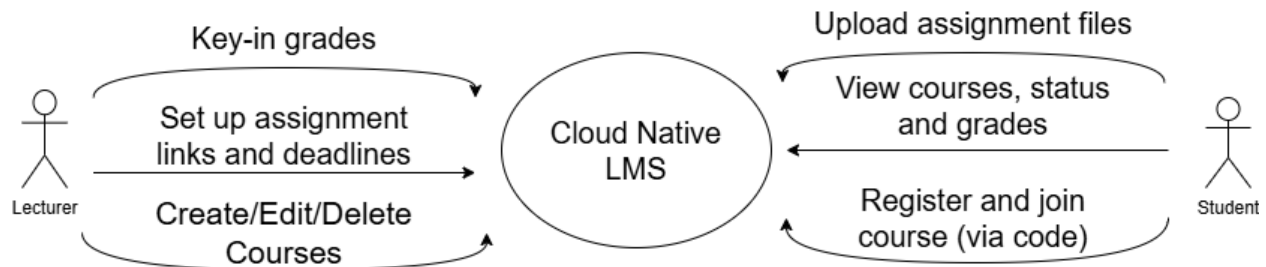


Diagram 1.0: Context Diagram

The context diagram shows how external users interact with the SMARTLEARN system at a high level. The main users are **students** and **lecturers**, who communicate directly with the system.

Students use the system to register, log in, enroll in courses, submit assignments, and view their grades. Lecturers use the system to create and manage courses, upload assignments, and evaluate student submissions.

The SMARTLEARN system acts as the central system that processes all requests and returns the results to users. This diagram helps to clearly define the system boundary and shows how data flows between users and the system without showing internal technical details.

11.2 Component/Service Architecture Diagram

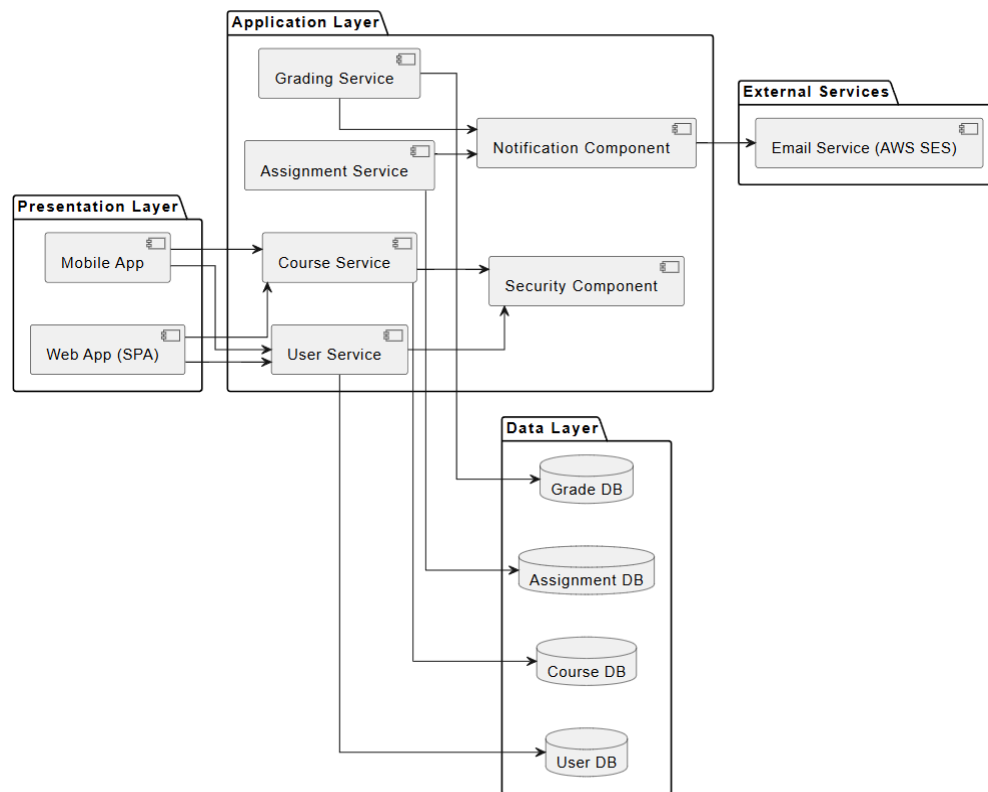


Diagram 2.0: Component/Service Diagram

The diagram illustrates the component and service architecture of the SMARTLEARN system based on a cloud-native 3-tier layered architecture. The system is divided into three main layers, namely the Presentation Layer, Application Layer, and Data Layer, with additional integration to external services. The Presentation Layer consists of the Web Application (SPA) and Mobile Application, which act as the user interface for students and lecturers to interact with the system. These applications send user requests to the Application Layer through REST API communication.

The Application Layer represents the core processing unit of the system, where all business logic is handled. It is composed of four main services, namely User Service, Course Service, Assignment Service, and Grading Service. Each service is responsible for a specific function, such as managing users, handling course operations, processing assignment submissions, and managing grading activities. In addition to these core services, the system includes supporting components such as the Security Component and Notification Component. The Security Component is responsible for handling authentication and authorization, ensuring that only authorized users can access system features. Meanwhile, the Notification Component manages system notifications, such as sending alerts for assignment submissions and grading updates.

The Data Layer stores all system data, including the User Database, Course Database, Assignment Database, and Grade Database. Each database is used by its respective service to store and retrieve relevant information. This design helps ensure better organization of data and improves system scalability. Furthermore, the Notification Component is integrated with an external service, specifically the Email Service (AWS SES), to deliver email notifications to users.

Overall, the architecture demonstrates a well-structured and scalable system where user requests flow from the Presentation Layer to the Application Layer for processing, and then to the Data Layer for storage, before responses are returned to the user interface. This design supports modularity, maintainability, and efficient system performance in a cloud environment.

11.3 Interaction Diagram

11.3.1 User Management Module

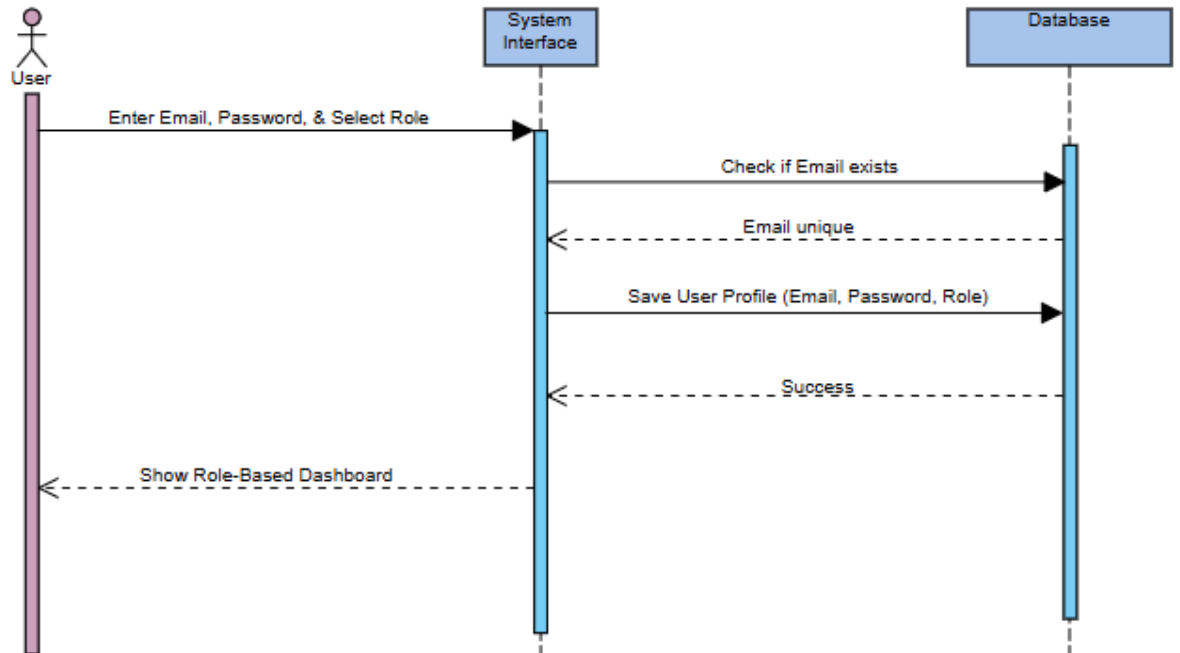


Diagram 3.1 Interaction Diagram (User Management Module)

The interaction diagram for the User Management Module illustrates a sequential process where the user first initiates the workflow by entering their email, password, and selected role into the system interface, which then triggers a validation request to the database to ensure the email address is unique. Once the database confirms the email's uniqueness, the system interface sends a secondary command to save the complete user profile into the database records, and upon receiving a successful confirmation from the database, the system interface concludes the cycle by presenting the user with a customized, role-based dashboard.

11.3.2 Course Management Module

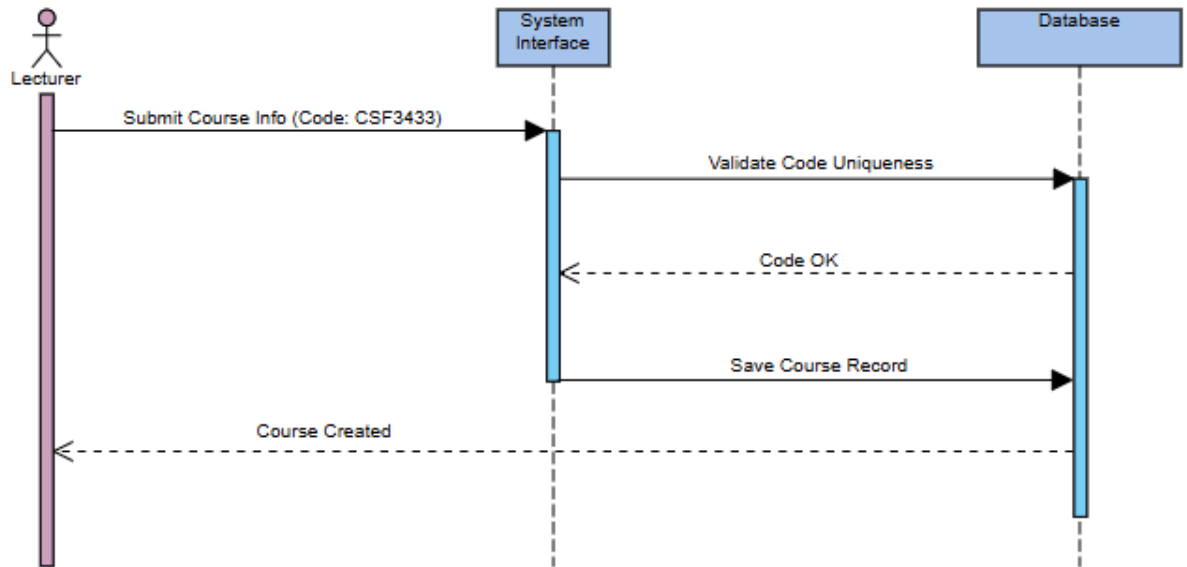


Diagram 3.2 Interaction Diagram (Course Management Module)

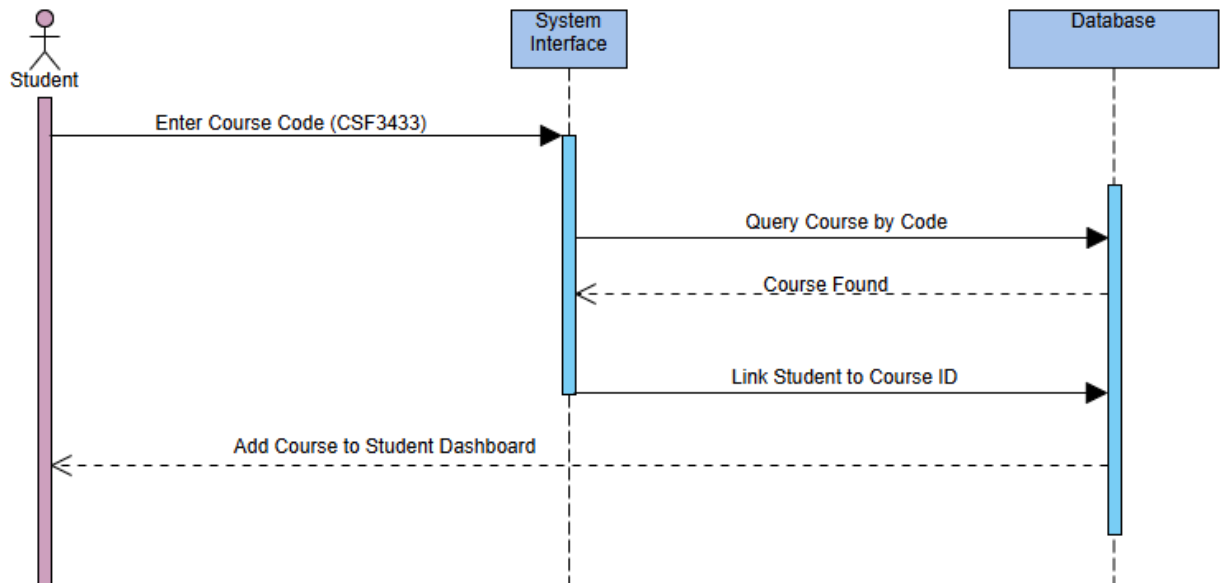


Diagram 3.2 Interaction Diagram (Course Management Module)

The Course Management Module interaction diagram is divided into two main workflows. First, a lecturer initiates the creation of a new course by submitting course information through the system interface, which then prompts the database to validate the

uniqueness of the course code before saving the record and confirming to the lecturer that the course has been created.

Second, a student enters a specific course code into the interface, triggering the system to query the database for that code and, upon finding the course, link the student's ID to the course record so that the course is successfully added to the student's personal dashboard.

11.3.3 Assignment Management Module

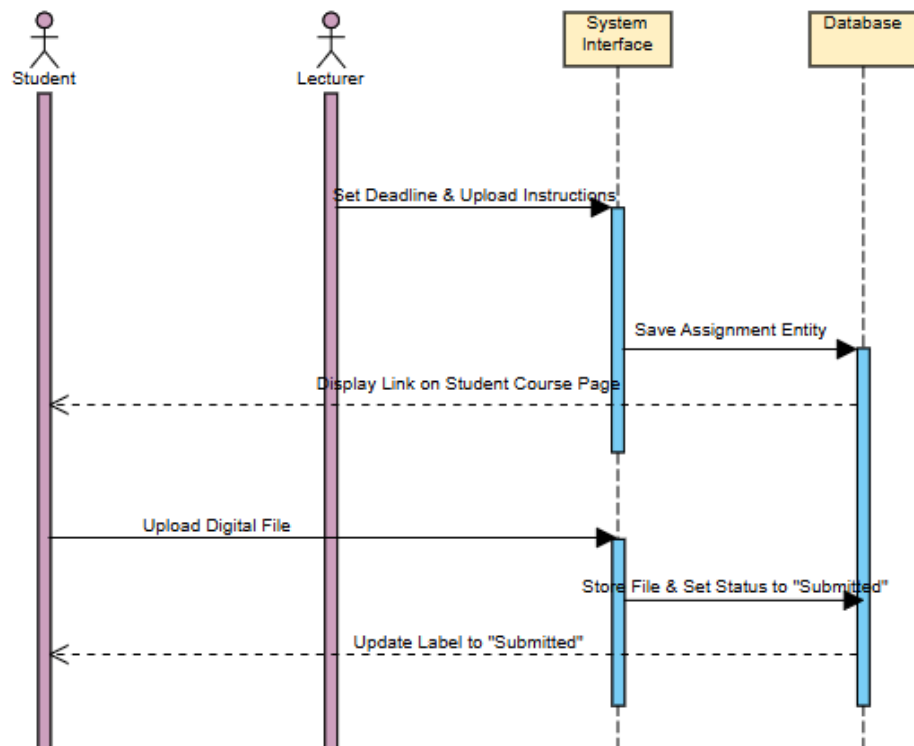


Diagram 3.3 Interaction Diagram (Assignment Management Module)

The Assignment Management Module interaction diagram depicts a two-phase process starting with a lecturer setting a deadline and uploading instructions through the system interface, which then saves the assignment entity into the database and displays a submission link on the student's course page. Subsequently, the student uploads their digital file to the system interface, which triggers the database to store the file and update the assignment status to "Submitted" while simultaneously updating the visual label on the student's interface to reflect that the submission is complete.

11.3.4 Grading Module

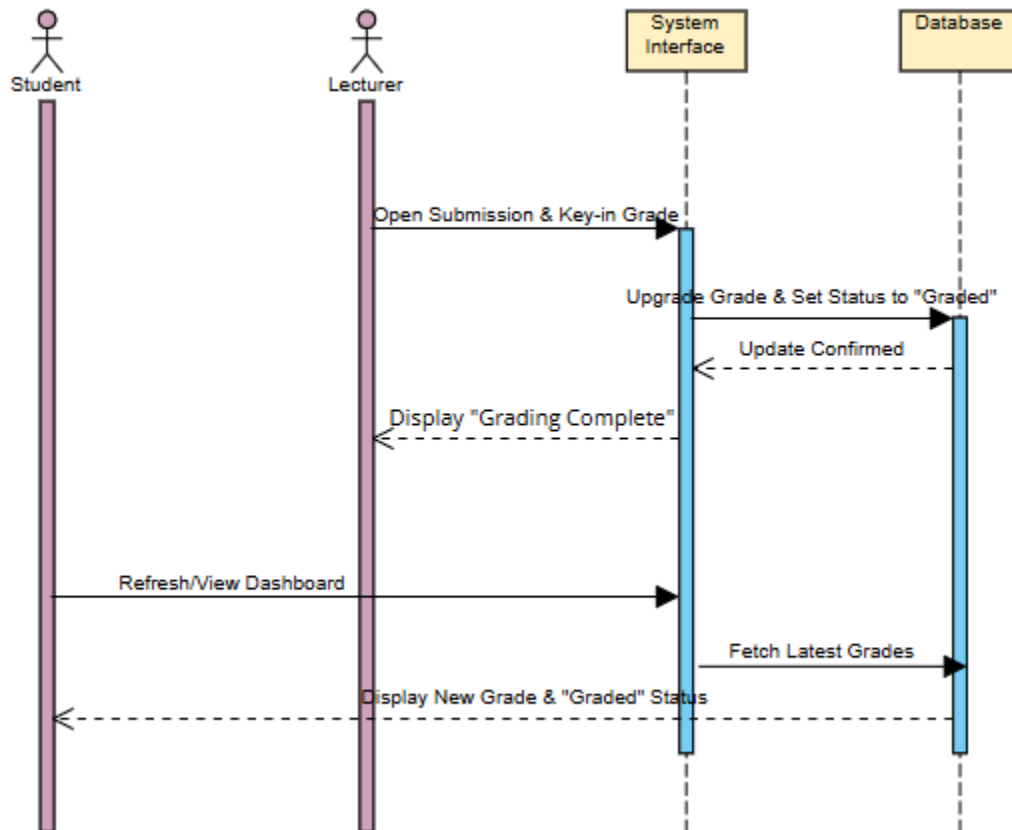


Diagram 3.4 Interaction Diagram (Grading Management Module)

The Grading Module interaction diagram shows a workflow where a lecturer first opens a student's submission and enters a grade through the system interface, which then updates the grade and sets the assignment status to "Graded" within the database. Once the database confirms the update, the lecturer receives a notification that grading is complete, and the student can subsequently refresh their dashboard to trigger the system interface to fetch the latest scores from the database and display the new grade along with the updated status.

11.4 Deployment Diagram

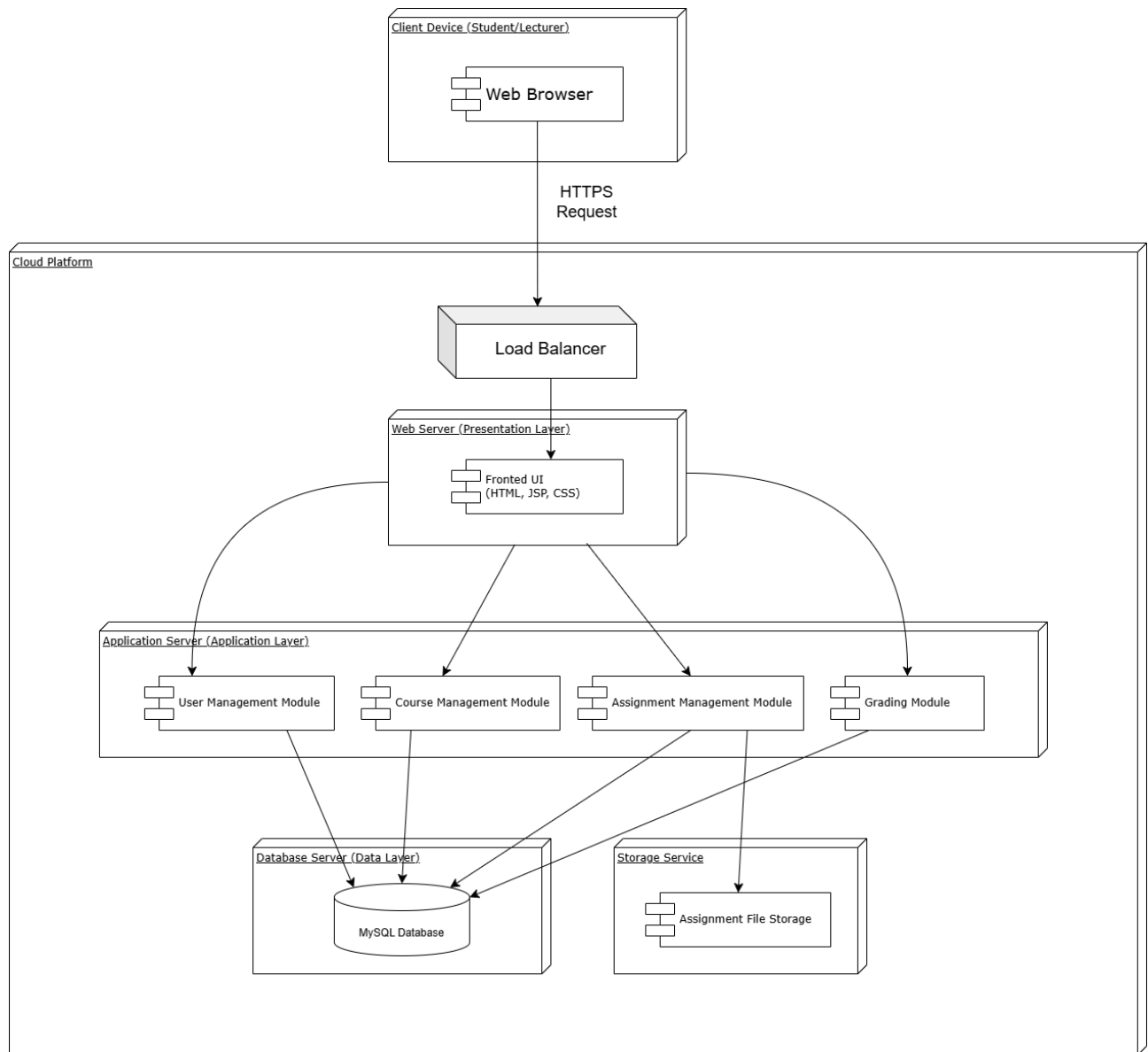


Diagram 4.0 Deployment Diagram

The deployment diagram shows how the SMARTLEARN system is physically deployed in a cloud environment. Users (students and lecturers) access the system through a web browser on their devices using an internet connection.

The system is hosted on a cloud platform and uses a **load balancer** to distribute incoming requests across servers. This helps the system handle many users at the same time without slowing down.

The system follows a **3-tier architecture**:

- The **Presentation Layer** (web server) handles the user interface
- The **Application Layer** (application server) processes system logic and modules
- The **Data Layer** (database server) stores all system data

Assignment files are stored in a separate storage service. This setup improves system performance, scalability, and availability, ensuring that the system can support many users and remain accessible at all times.

12.0 Team Member Roles

No.	Name	Role	Responsibilities
1.	Puteri Ireene Hazwani binti Muhamad Haizar	Frontend Developer	Building the user interface (UI) and ensuring a smooth user experience (UX)
2.	Nurul Alessa Binti Mohd Nizam	Backend Developer	Developing the business logic, security, and API endpoints
3.	Nurain Nazirah Binti Abdullah	Full-stack Architect & Database Lead	Designing the database structure and overseeing how the whole system fits together in the cloud.